FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress 44%

Go Back

Back To Practice

Q1

Save

Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>
struct Process {
    int id, at, bt, priority;
    int ct, tat, wt, completed;
};
int main() {
    int n;
    printf("Enter number of processes: \n");
    if (scanf("%d", &n) != 1) return 0;
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("Enter AT, BT and Priority for P%d: \n", p[i].id);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].priority);
        p[i].completed = 0;
    }
    int current_time = 0, completed_count = 0;
    float total_tat = 0, total_wt = 0;
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress 44%

Go Back

Back To Practice

Q1

Save

Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language : C (GCC 9.2.0)

```
        }
        if (idx != -1) {
            p[idx].ct = current_time + p[idx].bt;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;
            total_tat += p[idx].tat;
            total_wt += p[idx].wt;
            p[idx].completed = 1;
            completed_count++;
            current_time = p[idx].ct;
        } else {
            current_time++;
        }
    }
    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt / n);
    return 0;
}
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

31°C Search ENG 21:25